CCEE Exam: Java OOPS Module Cheatsheet

Comprehensive Review (Sessions 8 - 17)

Contents

1 Session 8: java.lang & java.util **2**
- 1.1 java.lang (The Foundation) . . . 2
- 1.2 java.util (The Toolkit) . . . 2

2 Sessions 9 & 10: Generics & Collections **2**
- 2.1 Generics . . . 2
- 2.2 Collection Interfaces . . . 2
- 2.3 Sorting . . . 3

3 Session 11: Functional Programming **3**
- 3.1 Functional Interface . . . 3
- 3.2 Lambda Expressions . . . 3
- 3.3 java.util.function Package . . . 3

4 Session 12: Streams API **3**
- 4.1 Streams vs Collections . . . 3
- 4.2 The Pipeline . . . 3
- 4.3 Java Date/Time (java.time) . . . 3

5 Sessions 13 & 14: Concurrency **4**
- 5.1 Creating Threads . . . 4
- 5.2 Lifecycle . . . 4
- 5.3 Synchronization . . . 4

6 Session 15: Java I/O **4**
- 6.1 Stream Types . . . 4
- 6.2 Serialization . . . 4

7 Session 16: JVM & Reflection **4**
- 7.1 JVM Architecture . . . 4
- 7.2 Reflection API . . . 5

8 Session 17: JDBC **5**
- 8.1 Architecture . . . 5
- 8.2 Driver Types . . . 5
- 8.3 Core Interfaces . . . 5

1 Session 8: java.lang & java.util

1.1 java.lang (The Foundation)

Note: Automatically imported.

- **Object Class:** Root of hierarchy. Methods: `toString()`, `equals()`, `hashCode()`, `getClass()`.
- **String (Immutable):** Once created, cannot change. Modifications create new objects.
- **StringBuilder/StringBuffer (Mutable):** Used for frequent string manipulation. `StringBuffer` is synchronized (thread-safe).
- **Wrapper Classes:** Encapsulate primitives (e.g., `int` $\rightarrow$ `Integer`). Allows primitives in Collections.
- **Autoboxing:** Automatic conversion between primitive and wrapper.

1.2 java.util (The Toolkit)

Note: Must be explicitly imported.

- Contains the Collections Framework, Date/Time, Random, Scanner.

```
String s1 = new String("Java"); // Heap Object
String s2 = "Java";             // String Pool
// s1 == s2 is FALSE (different addresses)
// s1.equals(s2) is TRUE (same content)
```

Listing 1: String Equality Trick

2 Sessions 9 & 10: Generics & Collections

2.1 Generics

Enables compile-time type safety.

- **Type Erasure:** Generics exist only at compile time. At runtime, types are replaced by `Object`.
- **Wildcards:**
  - `<?>`: Unbounded.
  - `<? extends T>`: Upper Bounded (Read-only, accepts T or children).
  - `<? super T>`: Lower Bounded (Write-allowed, accepts T or parents).

2.2 Collection Interfaces

- **List (Ordered, Duplicates OK):** `ArrayList` (Fast access), `LinkedList` (Fast insert/delete).
- **Set (Unique, Unordered):** `HashSet` (Fast, no order), `TreeSet` (Sorted, slower).
- **Map (Key-Value, Unique Keys):** `HashMap`, `TreeMap`.

2.3 Sorting

- **Comparable (java.lang):** 'Natural' ordering. Implement in the class itself via `compareTo()`.
- **Comparator (java.util):** Custom ordering. Separate class/lambda via `compare()`.

3 Session 11: Functional Programming

3.1 Functional Interface

An interface with exactly **one abstract method**. Can have multiple `default`/`static` methods.

3.2 Lambda Expressions

Syntax: `(params) -> { body }`

3.3 java.util.function Package

- **Predicate<T>:** `boolean test(T t)`. Used for filters.
- **Consumer<T>:** `void accept(T t)`. Used for printing/saving.
- **Supplier<T>:** `T get()`. No input, supplies output.
- **Function<T,R>:** `R apply(T t)`. Transforms input T to output R.

4 Session 12: Streams API

4.1 Streams vs Collections

- **Collection:** Stores data. Eager loading.
- **Stream:** Computes data. Lazy loading (nothing happens until terminal op). Immutable.

4.2 The Pipeline

1. **Source:** `list.stream()`
2. **Intermediate (Lazy):** `filter()`, `map()`, `sorted()`, `flatMap()` (flattens nested lists).
3. **Terminal (Eager):** `collect()`, `forEach()`, `reduce()`, `count()`.

4.3 Java Date/Time (java.time)

Immutable and Thread-safe (unlike old `Date`).

- `LocalDate` (Date only), `LocalTime` (Time only).
- `LocalDateTime` (Both).
- `LocalDate.now()` gets system time (not UTC by default).

5 Sessions 13 & 14: Concurrency

5.1 Creating Threads

- **Extend Thread:** `class MyThread extends Thread`.
- **Implement Runnable (Preferred):** `class MyTask implements Runnable`. Allows extending other classes.

5.2 Lifecycle

New → Runnable → Running → Blocked/Waiting → Terminated.

5.3 Synchronization

- **Race Condition:** Multiple threads modifying shared data concurrently.
- **synchronized:** Acquires an intrinsic lock (Monitor) on the object.
- **wait():** Releases lock and waits.
- **notify():** Wakes up a waiting thread.
- **sleep():** Pauses thread but **holds the lock**.

6 Session 15: Java I/O

6.1 Stream Types

- **Byte Streams (8-bit):** `InputStream`, `OutputStream`. Used for binary (Images, PDF).
- **Character Streams (16-bit):** `Reader`, `Writer`. Used for Text/Unicode.

6.2 Serialization

- Mechanism to save object state to a file.
- Class must implement `Serializable` (Marker interface).
- **transient:** Keyword to skip fields during serialization (e.g., passwords).

7 Session 16: JVM & Reflection

7.1 JVM Architecture

- **Method Area:** Stores class structures, static variables (Shared).
- **Heap:** Stores Objects (Shared, GC happens here).
- **Stack:** Stores method calls and local primitive variables (Thread-safe, auto-cleared).

7.2 Reflection API

Inspect code at runtime.

- **Get Class:** `obj.getClass()` or `Class.forName("Name")`.
- **Get Methods:** `getMethods()` (Public only), `getDeclaredMethods()` (All, including private).
- **Access Private:** `field.setAccessible(true)`.

8 Session 17: JDBC

8.1 Architecture

App $\rightarrow$ JDBC Interface $\rightarrow$ Driver Manager $\rightarrow$ Driver $\rightarrow$ Database.

8.2 Driver Types

- **Type 1:** JDBC-ODBC Bridge (Obsolete).
- **Type 4:** Thin Driver. Pure Java. Communicates directly with DB protocol. Standard choice.

8.3 Core Interfaces

- **Connection:** Session with DB.
- **Statement:** Compiles SQL every time. Vulnerable to SQL Injection.
- **PreparedStatement:** Pre-compiled (Cached). **Prevents SQL Injection**. Faster for repeated queries.
- **ResultSet:** Holds data. Cursor starts *before* first row. Use `next()` to iterate.

```
String sql = "SELECT * FROM Users WHERE id = ?";
try (Connection conn = DriverManager.getConnection(url, user, pass);
     PreparedStatement ps = conn.prepareStatement(sql)) {

    ps.setInt(1, 101); // Set param
    ResultSet rs = ps.executeQuery();

    while(rs.next()) {
        System.out.println(rs.getString("name"));
    }
} // Auto-closes resources
```

Listing 2: JDBC Best Practice